UADE Beats: Iteración 4

Grupo 9. Proceso de Desarrollo de Software (Análisis y Diseño Orientado a Objetos)
Rama: `iteracion-4` · Repositorio: github.com/lferraro1103/Proceso-de-desarrollo-de-sosftware

---

1. Qué es UADE Beats

UADE Beats es un sistema para **acceder a eventos en vivo** (recitales transmitidos por internet). Hay tres roles:

Rol	Qué hace
Usuario	Ve los eventos, pide ingresar a uno (según su plan de suscripción y la capacidad) y mira la transmisión.
Artista	Crea sus eventos, los inicia, pausa, reanuda, finaliza o cancela, y ve quién está conectado.
Administrador	Puede hacer todo: dar de alta usuarios y artistas, activar o desactivar cuentas, expulsar usuarios y ver todos los registros de acceso.

El programa está hecho en **Java** (ventana Swing y modo consola), con un servicio aparte en **Node.js** para las transmisiones.

---

2. De dónde partimos (iteración 3)

La iteración 3 aplicó los cinco principios **SOLID** y dejó armada la arquitectura **MVC** y el sistema de **roles y permisos**:

Principio	Qué se hizo en la iteración 3
SRP	Se sacó de `GestorEventosEnVivo` la lógica de ingresos a una clase nueva, `ControlAcceso`.
OCP	Se eliminaron los `instanceof RecitalEnVivo`: `Evento` declara las operaciones y los clientes trabajan con `Evento`.
LSP	Cualquier subclase de `Evento` puede usarse en el gestor y en las vistas.
ISP	La interfaz `Persistencia` tiene solo las dos operaciones que el gestor necesita.
DIP	El gestor depende de `Persistencia` (interfaz) y la recibe por constructor, en lugar de crear `PersistenciaArchivos`.

Los datos se guardaban en **archivos TXT** y todo corría en **una sola PC**.

---

3. Qué se hizo en la iteración 4 (resumen)

La iteración 4 tiene dos ejes:

1. **Patrones de diseño (tema de la clase 9 y 10):** se aplicó el patrón **Strategy** a la visibilidad de eventos y se documentó el Strategy que ya existía en los calendarios. También se revisaron Facade, Composite y Decorator para decidir si correspondía usarlos.

2. **Evolución del sistema:** base de datos SQLite, servidor con API HTTP, modo cliente remoto, contraseñas con hash, calendarios de Google y Apple y transmisiones.

Cambio	Antes (iteración 3)	Ahora (iteración 4)
Strategy de visibilidad	Un `if` dentro de `GestorEventosEnVivo.esVisiblePara` decidía qué eventos ve cada rol.	Una interfaz `PoliticaVisibilidadEvento` y dos estrategias (`VisibilidadGeneral` , `VisibilidadArtista`), elegidas por rol en `PoliticasVisibilidad` .
Base de datos	Archivos TXT	**SQLite**, con un repositorio por tipo de dato y cada guardado en una transacción.
Servidor	No había	**API HTTP con JSON**. Cada pedido lleva un *token* de sesión y el servidor filtra lo que ve cada rol.
Modo cliente	No había	Otra PC corre solo la interfaz y se conecta al servidor (`GestorRemoto`).
Vistas	Dependían del gestor concreto	Dependen de la interfaz `ServicioEventos` : funcionan igual con un gestor local o uno remoto.
Guardado	Botones de "guardar" y "cargar"	Automático: el gestor guarda después de cada operación.
Contraseñas	Texto plano	**Hash PBKDF2** con sal aleatoria por cuenta.
Calendarios	No había	Suscribir **Google Calendar** o **Apple Calendar** a un evento. El servidor de calendarios arranca solo con el programa.
Transmisiones	No había	Microservicio Node.js que busca un recital del artista y lo muestra como si fuera en vivo.
Avisos	`NotificadorNtfy` (push con ntfy.sh)	Se eliminó ntfy. Queda la interfaz `Notificador` con `NotificadorInactivo` .
Paquetes	7 paquetes	Se reordenó el código en paquetes por responsabilidad (`puertos` , `seguridad` , `servicio` , `servidor` , `cliente` , etc.).

4. Patrón Strategy

4.1 Qué es (según la cátedra)

Strategy es un **patrón de comportamiento**. Sirve para:

- **Encapsular** distintas formas (algoritmos) de resolver **el mismo problema** en clases diferentes.
- **Intercambiar** qué algoritmo se usa, sin que la clase que lo usa tenga que saber cuál es.

Se usa cuando un objeto tiene que hacer **la misma acción de formas distintas**, o cuando esa forma se elige **en tiempo de ejecución**.

Tiene tres participantes:

Participante	Qué es
Strategy (interfaz)	Define la firma del método que resuelve el problema.

Participante	Qué es
Estrategias concretas	Clases que implementan la interfaz, cada una con su algoritmo.
Contexto	Clase que tiene una referencia a la interfaz y **delega** en ella, usándola de forma polimórfica.

En GRASP se apoya en **Polimorfismo** ("que decida el objeto, no el if") y **Variaciones Protegidas**. En SOLID se relaciona con **OCP** y **DIP**.

En el proyecto hay **dos** lugares con Strategy: uno que **ya existía** antes del trabajo de patrones y uno que **se aplicó** en esta iteración.

4.2 Dónde ya estaba aplicado: vincular un calendario (Google / Apple)

> **Ya existía.** Se agregó junto con la función de calendarios (commit `db81aa3`, "Agrega la vinculación de Google y Apple Calendar"), antes de la revisión de patrones. En la iteración 4 **no se cambió**: se reconoció como Strategy, se movió la interfaz al paquete `puertos`, se documentó y se agregaron pruebas.

El problema: suscribir un calendario externo a un evento. La tarea es siempre la misma ("abrí tu calendario suscripto a esta dirección"), pero **cada proveedor lo hace distinto**:

- **Google:** arma un enlace `https://calendar.google.com/calendar/r?cid=...` y abre el navegador.
- **Apple:** convierte la dirección a `webcal://`. En Mac abre directamente la app Calendario; en otros sistemas se la entrega al sistema operativo.

Participante	Clase	Archivo
Strategy (interfaz)	`VinculoCalendario` con `getNombre()` y `vincular(urlFeed)`	`src/puertos/VinculoCalendario.java`
Estrategia concreta	`VinculoGoogle`	`src/calendario/VinculoGoogle.java`
Estrategia concreta	`VinculoApple`	`src/calendario/VinculoApple.java`
Contexto (local)	`GestorEventosEnVivo.vincularCalendario(idEvento, vinculo)`	`src/gestor/GestorEventosEnVivo.java`
Contexto (remoto)	`GestorRemoto.vincularCalendario(idEvento, vinculo)`	`src/cliente/GestorRemoto.java`
Quién elige la estrategia	El botón de la ventana y el menú de la consola	`ui/VentanaPrincipal.java`, `consola/MenuAcciones.java`

Cómo funciona:

```
Usuario toca "Google Calendar"
   VentanaPrincipal ──► vincularCalendario(new VinculoGoogle())
                                  │
                                  ▼
   gestor.vincularCalendario(idEvento, vinculo)
        1. verifica el permiso VINCULAR_CALENDARIO
        2. verifica que el evento sea visible para la cuenta
        3. arma la dirección del calendario del evento
        4. vinculo.vincular(url)   ◄── no sabe si es Google o Apple
```

```
// En la vista: se elige la estrategia
() -> vincularCalendario(new VinculoGoogle())
() -> vincularCalendario(new VinculoApple())

// En el gestor (contexto): se delega sin preguntar cuál es
public synchronized void vincularCalendario(int idEvento, VinculoCalendario vinculo)
        throws IOException {
    vinculo.vincular(urlCalendario(idEvento));
}
```

Detalle a tener en cuenta: el contexto recibe la estrategia **por parámetro** en cada llamada en vez de guardarla en un atributo. Sigue siendo Strategy: el algoritmo es intercambiable y el gestor no tiene ningún `if (google) ... else (apple)`. Si mañana se agrega Outlook, se crea `VinculoOutlook` y un botón, **sin tocar el gestor** (OCP).

Por qué en el remoto también sirve: el `GestorRemoto` le pide la dirección al servidor y después ejecuta **la misma estrategia** en la PC del cliente, que es donde está el navegador o la app de calendario.

Antecedente en la iteración 3: `Notificador`

En la iteración 3 ya había otra interfaz con dos implementaciones intercambiables: `Notificador`, con `NotificadorNtfy` (avisos push) y `NotificadorInactivo` (no hace nada). En la iteración 4 **se eliminó ntfy** y quedó solo `NotificadorInactivo`, que funciona como *Null Object*. Por eso hoy no se presenta como una familia de estrategias: con una sola implementación es un **punto de extensión** (inversión de dependencias), igual que `Persistencia` y `ProveedorTransmision`.

4.3 Dónde se aplicó en esta iteración: visibilidad de eventos

> **Es lo nuevo de la iteración 4** (commit `69fc9e8`, "Actualiza iteración 4 con Strategy y diagrama de clases completo").

El problema: decidir **qué eventos puede ver cada cuenta**.

- **Usuario y administrador** ven todo el catálogo.
- **Artista** ve solo los eventos en los que participa.

Esta regla se usa en muchos lugares: el listado de eventos, la búsqueda de un evento por ID, el estado que el servidor le manda a un cliente remoto y la autorización del calendario.

Antes: un `if` por rol dentro del gestor

```
// GestorEventosEnVivo (antes)
private boolean esVisiblePara(Cuenta actual, Evento evento) {
    if (actual == null) {
        return false;
    }
    if (actual.getTipoUsuario() != TipoUsuario.ARTISTA) {
        return true;
    }
    for (Artista artista : evento.getArtistas()) {
        if (artista.getId() == actual.getId()) {
            return true;
        }
    }
    return false;
}
```

Problemas:

- El gestor, que ya es una clase grande, **además** contenía el algoritmo de alcance por rol (baja cohesión).
- Para cambiar o agregar una regla había que **editar el gestor** (va contra OCP).
- No se podía probar la regla sin armar el gestor completo.

Después: interfaz + dos estrategias + contexto



Participante	Clase	Qué hace
Strategy (interfaz)	`PoliticaVisibilidadEvento`	`esVisible(Cuenta, Evento)`
Estrategia concreta	`VisibilidadGeneral`	Devuelve `true` si hay cuenta y evento (catálogo completo).
Estrategia concreta	`VisibilidadArtista`	Recorre los artistas del evento y compara por ID.
Contexto	`PoliticasVisibilidad`	Tiene un `Map<TipoUsuario, PoliticaVisibilidadEvento>` y elige la estrategia según el rol.
Cliente	`GestorEventosEnVivo`	Tiene un atributo `visibilidad` y le delega la decisión.

Todos en `src/gestor/`.

```
// Interfaz Strategy
public interface PoliticaVisibilidadEvento {
    boolean esVisible(Cuenta cuenta, Evento evento);
}

// Contexto: elige la estrategia por rol
public final class PoliticasVisibilidad {
    private final Map<TipoUsuario, PoliticaVisibilidadEvento> porRol = Map.of(
            TipoUsuario.USUARIO,       new VisibilidadGeneral(),
            TipoUsuario.ADMINISTRADOR, new VisibilidadGeneral(),
            TipoUsuario.ARTISTA,       new VisibilidadArtista());

    public boolean esVisible(Cuenta cuenta, Evento evento) {
        if (cuenta == null || evento == null) return false;
        PoliticaVisibilidadEvento politica = porRol.get(cuenta.getTipoUsuario());
        return politica != null && politica.esVisible(cuenta, evento);
    }
}

// GestorEventosEnVivo (después): una sola línea
private boolean esVisiblePara(Cuenta actual, Evento evento) {
    return visibilidad.esVisible(actual, evento);
}
```

Por qué dos estrategias y no tres: usuario y administrador tienen exactamente la misma regla, así que comparten `VisibilidadGeneral`. Hacer una clase por rol hubiera sido duplicar código.

Qué NO cambió (comportamiento conservado):

1. Sin cuenta o sin evento no se ve nada.
2. Usuario y administrador siguen viendo todo el catálogo.
3. Un artista ajeno no ve el evento aunque sepa su ID.
4. Un artista cargado de nuevo desde la base (otro objeto, mismo ID) sigue viendo sus eventos, porque se compara por ID.
5. **Ver no es lo mismo que poder modificar:** la visibilidad dice *qué se ve*; los permisos (`Permisos`, `ControlSesion`) siguen decidiendo *qué se puede hacer*.
6. La API, los DTO y la base de datos no cambiaron.

Qué se gana:

- **Alta cohesión / SRP:** el gestor coordina; la regla de alcance vive en su propia clase.
- **Polimorfismo:** el gestor no pregunta el rol; le pide a la estrategia que decida.
- **Pruebas:** las dos estrategias se prueban sin levantar servidor ni ventana.
- **Un solo lugar** para cambiar la regla, y lo usan listados, búsquedas, la API y los calendarios.

Honestidad del diseño (para la defensa): agregar un rol nuevo con una regla distinta requiere una clase nueva **y** una línea en el mapa de `PoliticasVisibilidad`. El gestor no se toca. El `if` anterior también era válido para un caso tan chico: se eligió Strategy porque la regla se usa en muchos lugares y conviene probarla aislada, no porque todo `if` necesite un patrón.

4.4 Comparación rápida de los dos Strategy

	Calendarios	Visibilidad
¿Cuándo?	Ya existía (antes de la revisión de patrones)	Aplicado en la iteración 4

	Calendarios	Visibilidad
Interfaz	`VinculoCalendario`	`PoliticaVisibilidadEvento`
Estrategias	`VinculoGoogle`, `VinculoApple`	`VisibilidadGeneral`, `VisibilidadArtista`
¿Quién elige?	La persona (botón o menú)	El sistema, según el rol de la cuenta
¿Cómo llega al contexto?	Por parámetro en cada llamada	Mapa fijo por rol dentro de `PoliticasVisibilidad`
Qué evita	Un `if (google) / else (apple)` en el gestor	Un `if (rol == ARTISTA)` en el gestor

---

5. Otros patrones presentes

Patrón	Dónde	Para qué
MVC	Modelo (`modelo`), controlador (`GestorEventosEnVivo` detrás de `ServicioEventos`), vistas (`ui` , `consola`)	Separar datos, lógica y presentación. Viene de la iteración 3.
Adapter	`ClienteTransmision` implementa `ProveedorTransmision` y traduce cada llamada a un pedido HTTP al servicio Node.js. `VinculoGoogle` / `VinculoApple` también adaptan la API de escritorio de Java.	El gestor usa una interfaz propia y no sabe que del otro lado hay un servicio HTTP. Es la idea del Adapter visto en clase: "un enchufe para conectar dos piezas que no encajan".
Repository	`RepositorioUsuarios` , `RepositorioArtistas` , `RepositorioEventos` , `RepositorioRegistros`	Cada uno traduce entre una tabla SQL y los objetos del modelo. El resto del programa no conoce SQL.
Proxy remoto	`GestorRemoto`	Implementa `ServicioEventos` igual que el gestor local, pero reenvía cada pedido al servidor. Las vistas no saben cuál usan.
Facade	`GestorEventosEnVivo` (a través de `ServicioEventos`)	Ofrece los casos de uso y por dentro coordina `ControlSesion` , `ControlAcceso` , `ControlTransmision` , `Persistencia` y los calendarios.
DTO	Paquete `api`	Objetos que viajan por la red. Ninguno lleva contraseñas.
Null Object	`NotificadorInactivo`	Cumple la interfaz `Notificador` sin hacer nada.

Patrones revisados y descartados (a propósito):

Patrón	Decisión	Motivo
Composite	No se aplicó en el dominio	No hay eventos que contengan otros eventos. Solo se usa el Composite que ya trae Swing (paneles con componentes). Tendría sentido si aparecieran festivales con escenarios o subeventos.
Decorator	No se aplicó	No hay ninguna funcionalidad que se quiera "apilar" sobre una interfaz existente. Se evaluaron casos (métricas o reintentos en `Persistencia` , caché en `ServicioEventos`) y se pospusieron por falta de un requisito real.

Patrón	Decisión	Motivo
State	No se aplicó	`EstadoEvento` es un enum con reglas de transición simples. Seis clases State serían sobre-ingeniería.

---

6. Cómo funciona el sistema

6.1 Arquitectura por capas

```
┌──────────── Vistas ────────────┐
│  ui (Swing)        consola     │   dependen solo de ServicioEventos
└──────────────┬─────────────────┘
               │ ServicioEventos = ServicioSesion + ServicioConsultas + ServicioComandos
     ┌─────────┴──────────┐
     ▼                    ▼
GestorEventosEnVivo     GestorRemoto ──HTTP/JSON──► ServidorHttp ──► GestorEventosEnVivo
 (local, con datos)     (cliente, sin datos)                         (en la PC servidor)
     │
     ├── ControlSesion + Permisos ............ quién puede hacer qué
     ├── PoliticasVisibilidad (Strategy) ..... qué eventos ve cada uno
     ├── ControlAcceso ....................... reglas de ingreso a un evento
     ├── ControlTransmision ─► ProveedorTransmision ─► ClienteTransmision ─► servicio Node.js
     ├── Persistencia ─► PersistenciaSqlite ─► Repositorios ─► SQLite
     ├── PublicadorCalendario ─► PublicadorHttp
     └── Notificador ─► NotificadorInactivo
```

`Arranque` es el único lugar que crea las clases concretas (`PersistenciaSqlite` , `NotificadorInactivo` , `PublicadorHttp` , `ClienteTransmision`) y se las pasa al gestor por constructor. El gestor solo conoce interfaces (**DIP**).

6.2 Tres modos de ejecución

Modo	Comando	Qué hace
Completo	`Main`	Datos en SQLite, API (solo desde el mismo equipo), calendarios publicados en la red local y ventana o consola.
Servidor	`Main servidor`	Solo datos y API, sin interfaz. Acepta conexiones de la red.
Cliente	`Main cliente http://IP:8765`	Solo interfaz. No guarda nada: inicia sesión en el servidor y trabaja con lo que su rol puede ver.

6.3 Ejemplo de flujo: un usuario pide ingresar a un evento desde otra PC

1. La ventana llama `servicio.solicitarIngreso(...)` . En modo cliente, el servicio es un `GestorRemoto` .
2. `GestorRemoto` manda `POST /api/eventos/{id}/ingresos` con el token de la sesión.
3. El servidor abre la sesión de ese token y llama al `GestorEventosEnVivo` real.
4. El gestor verifica el **permiso**, que el evento sea **visible** (Strategy) y las **reglas de ingreso** (`ControlAcceso` : estado, plan, capacidad, cuenta activa).
5. Guarda el resultado en SQLite (dentro de una transacción) y lo devuelve.
6. Si el ingreso es rechazado, el servidor responde 409 y el cliente lanza la misma `AccesoDenegadoException` que lanzaría el gestor local. La vista no nota la diferencia.
7. Si el evento está en curso, se abre sola la página de la transmisión.

6.4 Base de datos

Un archivo SQLite (`~/.uadebeats/uadebeats.db`) con seis tablas: `usuarios` , `artistas` , `eventos` , `evento_artistas` , `evento_conectados` y `registros_acceso` . Si la base está vacía, se cargan datos de prueba (8 usuarios, 5 artistas, 7 eventos en distintos estados y 16 accesos).

6.5 Seguridad

- **Contraseñas:** PBKDF2 con HMAC-SHA256, 120.000 iteraciones y sal aleatoria de 16 bytes. Nunca se guardan ni se envían al cliente.
- **Sesiones remotas:** token aleatorio al iniciar sesión; la contraseña viaja una sola vez.
- **Filtrado por rol en el servidor:** el cliente recibe solo lo que su rol puede ver. Las reglas se aplican en un único lugar.

6.6 Calendarios y transmisiones

- **Calendarios:** el servidor publica cada evento en `/calendario/<código>/<id>.ics` . Google o Apple se suscriben a esa dirección y el evento se actualiza solo (si cambia el horario o se cancela). El servidor de calendarios **arranca solo con el programa**, sin ejecutar `Main servidor` :
  - En modo completo, un servidor aparte publica solo los `.ics` en la red local (puerto 8767). La API sigue accesible solo desde el mismo equipo. Si un puerto está ocupado se usa uno libre, y todo se cierra al salir.
  - La dirección se elige en este orden: `-Duadebeats.url` , el túnel de Cloudflare (si `cloudflared` está instalado, el programa lo abre solo hacia los calendarios) o la IP de la red local.
  - **Google** necesita https (túnel o `uadebeats.url`); si no lo hay, informa el motivo. **Apple** usa la https del túnel si existe, y si no `webcal://` sobre la red local. Esto último no está verificado: en una prueba anterior en Mac, Calendario convirtió `webcal` en `https` y la suscripción local no terminó.
- **Transmisiones:** un servicio Node.js busca en YouTube un recital del artista del evento y lo muestra a pantalla completa, arrancando en el minuto que corresponde según cuándo empezó el evento. Solo se puede ver con el evento en curso, y un usuario solo si ya ingresó. Se cierra si la persona cierra sesión, la expulsan o el evento termina.

6.7 Mejoras de la ventana

Eventos "● EN VIVO" en rojo, doble clic con acción según el rol, confirmaciones antes de acciones destructivas, actualización automática cada 5 segundos, buscador, barra de estado con colores y menos botones.

---

7. Relación con GRASP y SOLID

Concepto	Dónde se ve en la iteración 4
Polimorfismo (GRASP)	Las dos estrategias: el gestor no pregunta el rol ni el proveedor de calendario.
Variaciones Protegidas (GRASP)	`VinculoCalendario` , `PoliticaVisibilidadEvento` , `Persistencia` , `ProveedorTransmision` : lo que puede cambiar queda detrás de una interfaz.
Indirección (GRASP)	`GestorRemoto` y `ClienteTransmision` hacen de intermediarios con la red.
Fabricación Pura (GRASP)	Repositorios, `PoliticasVisibilidad` , `ControlTransmision` : clases que no son del dominio pero ordenan responsabilidades.

Concepto	Dónde se ve en la iteración 4
SRP	La regla de visibilidad sale del gestor; las reglas de transmisión pasan a `ControlTransmision`; los formularios salen de `VentanaPrincipal` a `Formularios`.
OCP	Un calendario nuevo es una clase nueva, sin tocar el gestor.
ISP	`ServicioEventos` se divide en `ServicioSesion`, `ServicioConsultas` y `ServicioComandos`.
DIP	El gestor recibe todas sus dependencias por constructor como interfaces; `PersistenciaSqlite` reemplazó a la de archivos sin tocar el gestor.

---

8. Verificación

- `Codigo/test/PruebasIteracion4.java`: según el repositorio, **110 comprobaciones** que cubren las estrategias, permisos, el servicio local, el remoto por HTTP y SQLite (se ejecutan con `java -ea`, sin JUnit).
- Las pruebas de Strategy verifican que un artista ajeno no vea un evento, que un artista recargado desde la base siga viendo los suyos y que la estrategia de calendario no se ejecute para un evento que la cuenta no puede ver.
- No se probaron servicios externos reales (YouTube, Google, Apple) en las pruebas automáticas.
- Diagrama de clases completo contrastado con el código: 91 archivos Java, 97 tipos.

---

9. Cómo ejecutarlo

Desde `TP/Grupo9_POO_EntregaFinal/Codigo` (en Windows el separador del classpath es `;`):

```
javac -encoding UTF-8 -cp "lib/*" -d out $(find src -name "*.java")
java --enable-native-access=ALL-UNNAMED -cp "out:lib/*" Main
java --enable-native-access=ALL-UNNAMED -cp "out:lib/*" Main servidor
java -cp "out:lib/*" Main cliente http://192.168.0.10:8765
```

Cuentas de prueba: `admin` / `admin123`; usuarios como `lferraro` / `lferraro2024`; artistas como `bandahorizonte` / `horizonte2024`. El evento 106 ("Sunrust - Transmisión permanente") siempre está en curso para la demo.

---

10. Limitaciones conocidas

Limitación	Detalle
Conexión sin cifrar	La API usa HTTP; fuera de una red de confianza conviene un túnel con https.
Sesiones	Viven en memoria y no vencen por tiempo.
Transacciones	Si falla el guardado en SQLite se deshace la base, pero los objetos en memoria ya se modificaron.
Concurrencia	La ventana del equipo servidor lee objetos sin bloqueo mientras un cliente remoto puede modificarlos.

Limitación	Detalle
Transmisiones	Sin clave de la API de YouTube se lee la página de resultados, que no es estable.

11. Preguntas probables en la defensa

¿Dónde aplicaron Strategy?
En la visibilidad de eventos: `PoliticaVisibilidadEvento` (interfaz), `VisibilidadGeneral` y `VisibilidadArtista` (estrategias), `PoliticasVisibilidad` (contexto) y `GestorEventosEnVivo` delega en él. Además ya existía Strategy en los calendarios: `VinculoCalendario` con `VinculoGoogle` y `VinculoApple`.

¿Qué había antes?
Un `if` por rol dentro de `GestorEventosEnVivo.esVisiblePara`. En calendarios no hubo un "antes" con `if`: se diseñó con Strategy desde que se agregó la función.

¿Por qué es Strategy y no simplemente herencia?
Porque el comportamiento que varía (el algoritmo) está separado del objeto que lo usa, y el contexto lo usa a través de una interfaz sin saber cuál es. No es un `Usuario` que sobrescribe un método: es un algoritmo aparte que se puede cambiar.

¿En calendarios, el contexto no guarda la estrategia en un atributo, sigue siendo Strategy?
Sí. Lo esencial es que el algoritmo sea intercambiable detrás de una interfaz y que el contexto no tenga condicionales por proveedor. Recibirla por parámetro es una variante válida.

¿Strategy y Adapter no son lo mismo? Los dos usan una interfaz.
No. Strategy ofrece **varias formas de hacer lo mismo** para poder elegir entre ellas. Adapter **traduce** una interfaz que no encaja a la que el cliente espera (por ejemplo, `ClienteTransmision` convierte llamadas Java en pedidos HTTP al servicio Node.js).

¿Ver un evento implica poder modificarlo?
No. La estrategia de visibilidad solo decide qué se ve. Qué se puede hacer lo deciden `Permisos` y `ControlSesion`.

¿Por qué no aplicaron Composite o Decorator?
Porque no hay un problema real que resuelvan hoy. Se documentó en qué caso sí tendrían sentido.